ByteMart Digital

THE COMPLETE GUIDE

Java 17

Modern Java: The Complete Guide to the LTS Release

A complete, example-driven guide to modern Java through the Java 17 LTS release — records, sealed classes, pattern matching, switch expressions, the module system, HttpClient, and migration.

24	120+	12	6
Chapters	Code Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Java 17: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Java, OpenJDK, and other product names, logos, and brands referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. Java 17 & the LTS Model 4
2. From Java 8 to 17 6
3. Local Variable Type Inference: var 9
4. Text Blocks & New String Methods 12
5. Records: Concise Data Carriers 15
6. Record Features & Customization 18
7. Records in Practice 21
8. Records vs Classes vs Lombok 23
9. Sealed Classes 25
10. Sealed Classes in Depth 27
11. Pattern Matching for instanceof 29
12. Pattern Matching for switch 32
13. Records + Sealed + Patterns 35
14. Switch Expressions 38
15. Collection Factory Methods 41
16. Stream Enhancements 43
17. Optional Improvements 45
18. Helpful NullPointerExceptions 47
19. The Module System (JPMS) 49
20. The Modern HttpClient 52
21. Enhanced Random & API Additions 55
22. Strong Encapsulation of Internals 57
23. Migrating from Java 8 to 17 59
24. Best Practices & Modern Java Style 61

MODULE 1 · THE MODERN JAVA LANDSCAPE

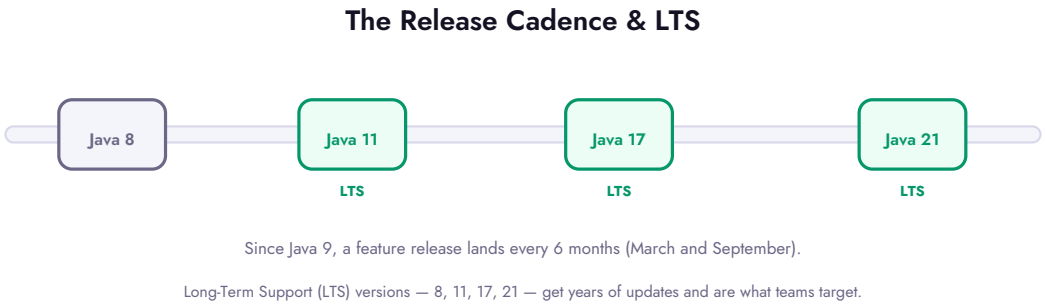
CHAPTER 01

Java 17 & the LTS Model

Why Java 17 matters, and how the six-month release cadence and LTS versions work.

Java 17, released in September 2021, is a **Long-Term Support** (LTS) release — the version most teams target for years. Understanding why it matters starts with understanding how Java is released today.

The release cadence



Feature releases land every six months; LTS versions (8, 11, 17, 21) get years of support.

Since Java 9 in 2017, a new feature release arrives every six months, in March and September. Most of these are short-lived. Every few years, one is designated **LTS** — 8, 11, 17, and 21 — receiving years of security and bug-fix updates.

Why LTS matters

- **Stability** — supported for years, safe to build on.
- **Adoption** — most libraries and employers target LTS versions.
- **Migration target** — teams jump from one LTS to the next (8 to 11 to 17).

Why 17 specifically

Java 17 gathers years of language and library evolution — records, sealed classes, pattern matching, text blocks, the modern HttpClient, and much more — into one stable, supported release. For a developer coming from Java 8, it is where all the modern features finally converge.

> **NOTE**
> The next LTS after 17 is Java 21 (2023), which finalizes features that are preview in 17 (like pattern matching for switch). This guide focuses on what is **final** in 17, noting preview features as they arise.

> **The version teams target**
> Java releases every six months, with LTS versions (8, 11, 17, 21) supported for years. Java 17 consolidates a decade of modern features into one stable LTS — the natural upgrade target from Java 8.

MODULE 1 · THE MODERN JAVA LANDSCAPE

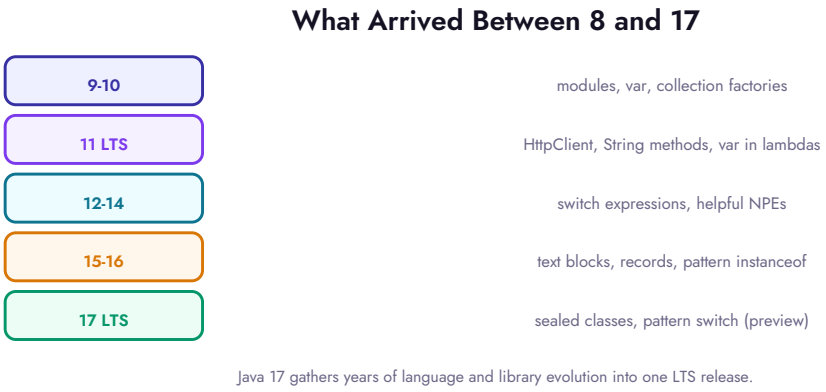
CHAPTER 02

From Java 8 to 17

A map of the major features added across the versions between the two big LTS releases.

Most developers know Java 8 — lambdas and streams. The leap to 17 crosses nine versions of steady evolution. This chapter maps the landmarks so you know what is new.

The journey



Java 17 gathers years of language and library evolution into one LTS release.

Version	Landmark additions
9-10	modules, var, List.of/Set.of/Map.of
11 LTS	HttpClient, new String methods
12-14	switch expressions, helpful NPEs
15-16	text blocks, records, instanceof patterns
17 LTS	sealed classes, pattern switch (preview)

Two kinds of change

The additions fall into two groups: **language features** (records, sealed classes, pattern matching, switch expressions, text blocks, var) that change how you write code, and **library additions** (HttpClient, String and Stream methods, collection factories) that give you better tools.

How to read this guide

The rest of the guide takes these features one theme at a time — records, then sealed classes and pattern matching, then the smaller language and library wins, then platform features, and finally migration. By the end you will write Java the modern way.

© 2026 ByteMart Digital

Nine versions of evolution

Between Java 8 and 17, each release added language features (records, sealed, patterns, switch, text blocks, var) and library tools (HttpClient, String/Stream methods, factories). Java 17 brings them together.

© 2026 ByteMart Digital

MODULE 1 · THE MODERN JAVA LANDSCAPE

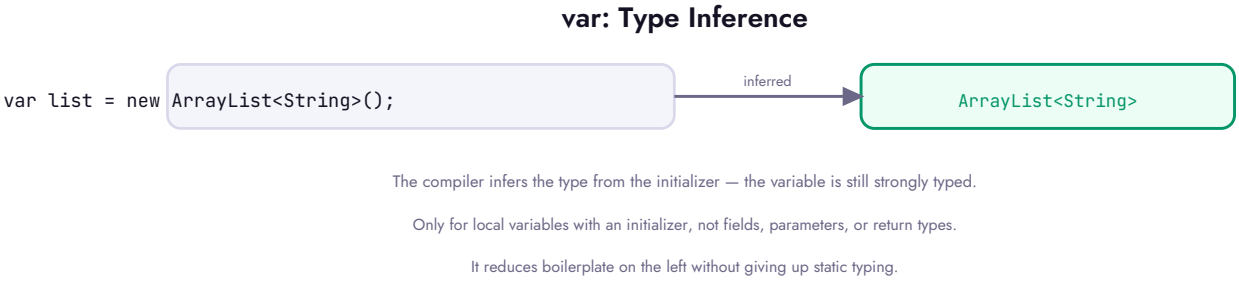
CHAPTER 03

Local Variable Type Inference: var

Letting the compiler infer local variable types to cut boilerplate without losing type safety.

Introduced in Java 10, `var` lets you declare a local variable without writing its type — the compiler infers it from the initializer. It reduces noise while keeping Java fully statically typed.

How it works



The compiler infers the type from the initializer; the variable is still strongly typed.

```java
// verbose
Map<String, List<Order>> ordersByUser = new HashMap<>();

// with var
var ordersByUser = new HashMap<String, List<Order>>();
// type is still Map<String, List<Order>> — just inferred
```

Still strongly typed

`var` is **not** dynamic typing. The variable has a fixed, compiler-known type from its initializer, and using it wrongly is still a compile error. It only saves you from writing the type twice.

Where var is allowed

Allowed	Not allowed
local variables	fields
for-loop variables	method parameters
try-with-resources	return types

WATCH OUT

`var` needs an initializer to infer from, so `var x = null;` and `var x;` do not compile. And use it where the type is obvious — `var x = getData();` can hurt readability if the return type is unclear.

Infer the obvious

var lets the compiler infer a local variable's type from its initializer, cutting boilerplate while keeping full static typing. Use it for local variables where the type is clear — not for fields, parameters, or returns.

© 2026 ByteMart Digital

MODULE 1 · THE MODERN JAVA LANDSCAPE

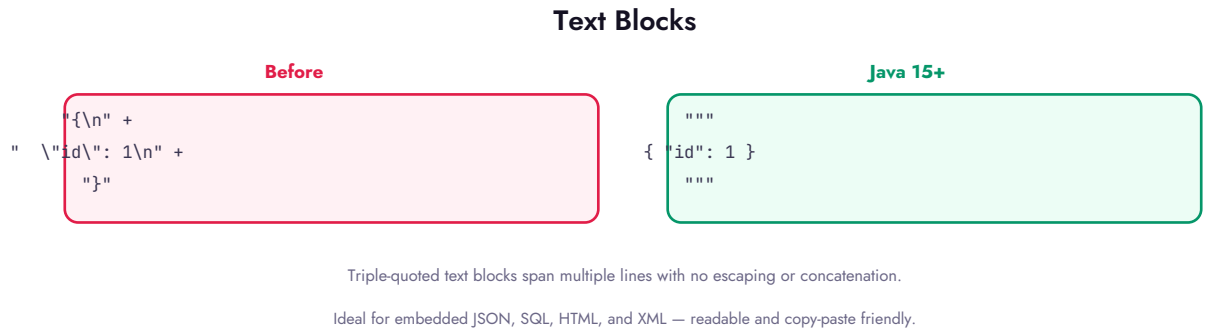
CHAPTER 04

Text Blocks & New String Methods

Multi-line string literals and the string utilities added since Java 8.

Working with multi-line text — JSON, SQL, HTML — was painful in Java 8, all escapes and concatenation. **Text blocks** (final in Java 15) fix that, and a set of new **String methods** round out the toolkit.

Text blocks



Triple-quoted text blocks span multiple lines with no escaping or concatenation.

```java
String json = """
    {
      "id": 1,
      "name": "Ada"
    }
    """;
// no escaped quotes, no + concatenation, real line breaks
```

A text block is delimited by triple quotes. It preserves line breaks, strips incidental leading whitespace based on the closing delimiter's indentation, and needs no escaped quotes — perfect for embedded snippets.

New String methods

Method	Does
isBlank()	true if empty or only whitespace
strip()	trim whitespace (Unicode-aware)
lines()	a Stream of the lines
repeat(n)	the string repeated n times
formatted(..)	like String.format on the instance

strip vs trim

`strip()` is the modern replacement for `trim()` — it understands Unicode whitespace, whereas `trim()` only handles ASCII. Prefer `strip()` in new code.

Readable text, richer strings

Text blocks give clean multi-line literals for JSON, SQL, and HTML with no escaping. New String methods — isBlank, strip, lines, repeat, formatted — round out everyday text handling; prefer strip over trim.

© 2026 ByteMart Digital

MODULE 2 · RECORDS & DATA CLASSES

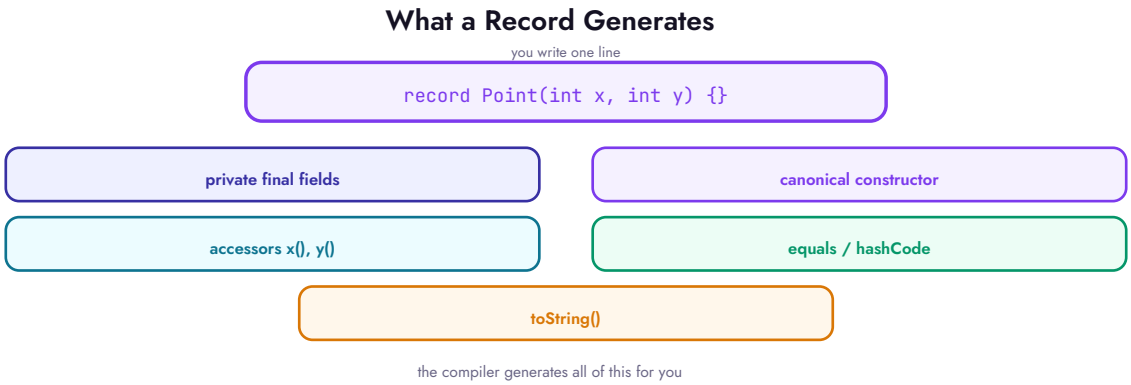
CHAPTER 05

Records: Concise Data Carriers

Replacing verbose data classes with a single declarative line.

Java was infamous for boilerplate: a simple data class meant fields, a constructor, getters, `equals`, `hashCode`, and `toString` — dozens of lines for a bag of data. **Records** (final in Java 16) collapse all of that into one line.

What a record generates



You write one line; the compiler generates fields, constructor, accessors, equals, hashCode, and toString.

```java
// the whole class:
record Point(int x, int y) {}

// usage
var p = new Point(3, 4);
p.x();              // 3  (accessor, not getX())
p.equals(new Point(3, 4));   // true
p.toString();       // Point[x=3, y=4]
```

What you get for free

- **Private final fields** for each component — records are immutable.
- **A canonical constructor** matching the components.
- **Accessor methods** named after the components (`x()`, not `getX()`).
- **equals, hashCode, toString** based on all components.

Immutability by design

A record's components are final — you cannot change them after construction. This makes records naturally safe to share across threads and ideal for value-like data. To 'change' one, create a new record.

Data classes in one line

A record declares an immutable data carrier in a single line, and the compiler generates the fields, constructor, accessors, equals, hashCode, and toString. It ends Java's classic data-class boilerplate.

© 2026 ByteMart Digital

MODULE 2 · RECORDS & DATA CLASSES

CHAPTER 06

Record Features & Customization

Validation, extra methods, and the flexibility records still allow.

Records are concise but not rigid. You can add validation, extra constructors, and methods — while the compiler still generates the boilerplate. Knowing these features lets you use records for real domain types.

The compact constructor

A **compact constructor** lets you validate or normalise components without restating the parameter list. It runs before the fields are assigned.

```java
record Range(int low, int high) {
    Range {                       // compact constructor
        if (low > high)
            throw new IllegalArgumentException("low > high");
    }
}
```

Adding methods and static members

Records can have extra instance methods, static factory methods, static fields, and can implement interfaces — everything a normal class can, except mutable state and inheritance.

```java
record Point(int x, int y) {
    static Point origin() { return new Point(0, 0); }
    double distanceTo(Point o) {
        return Math.hypot(x - o.x, y - o.y);
    }
}
```

The limits

Records can	Records cannot
implement interfaces	extend a class
add methods & statics	be extended (they are final)
validate in constructor	have mutable fields

Concise, still flexible

Records support compact constructors for validation, extra methods, static factories, and interfaces — while the compiler generates the boilerplate. They cannot inherit, be extended, or hold mutable state.

MODULE 2 · RECORDS & DATA CLASSES

CHAPTER 07

Records in Practice

Where records shine — DTOs, value objects, and multiple return values.

Records are not just a syntax trick — they change how you model data. Certain patterns become dramatically cleaner once you reach for records by default.

DTOs and API models

Request and response objects, API DTOs, and configuration holders are ideal records: they carry data, need value equality, and should be immutable. A record expresses exactly this intent in one line.

```java
record CreateUser(String name, String email) {}
record UserResponse(Long id, String name, String email) {}
// clean, immutable, self-documenting API contracts
```

Value objects

Domain values — money, coordinates, date ranges — benefit from records' immutability and value equality. Two `Money(100, "INR")` instances are equal, as value objects should be.

Multiple return values

Instead of an out-parameter or a loosely-typed array, return a small record. It names the parts and stays type-safe — far clearer than returning an `Object[]` or a two-field pair class.

```java
record MinMax(int min, int max) {}
MinMax bounds(int[] a) { /* ... */ return new MinMax(lo, hi); }
```

When not to use a record

WATCH OUT

Records are for data that is defined by its values. If your type has identity, mutable state, or a complex lifecycle (like a JPA entity), a regular class is the better fit — do not force records everywhere.

Reach for records for data

Use records for DTOs, API models, value objects, and multiple return values — anywhere data is defined by its values and should be immutable. Keep regular classes for types with identity or mutable state.

© 2026 ByteMart Digital

MODULE 2 · RECORDS & DATA CLASSES

CHAPTER 08

Records vs Classes vs Lombok

Choosing between records, hand-written classes, and the Lombok library.

Before records, many teams used **Lombok** to cut data-class boilerplate. Now there are three options, and knowing when to use each avoids both boilerplate and over-engineering.

The three approaches

Approach	Best for
record	immutable data carriers
plain class	identity, mutability, inheritance
Lombok	mutable beans, legacy entities

Records vs Lombok

For immutable data, records are now the better choice — they are a language feature (no annotation processor, no extra dependency, understood by every tool). Lombok's `@Value` did the same job, but records are built in.

Where Lombok still fits

Lombok remains useful for **mutable** classes — JPA entities, form-backing beans — where `@Getter`, `@Setter`, and `@Builder` reduce boilerplate that records cannot address (records are immutable and cannot be entities cleanly).

The decision

- Immutable data carrier? **Use a record.**
- Mutable bean or JPA entity? **A class, optionally with Lombok.**
- Needs inheritance or identity? **A plain class.**

Records first for data
For immutable data, prefer records — a built-in language feature with no dependency. Use plain classes (optionally with Lombok) for mutable beans, entities, and types needing identity or inheritance.

© 2026 ByteMart Digital

MODULE 3 · SEALED CLASSES & PATTERN MATCHING

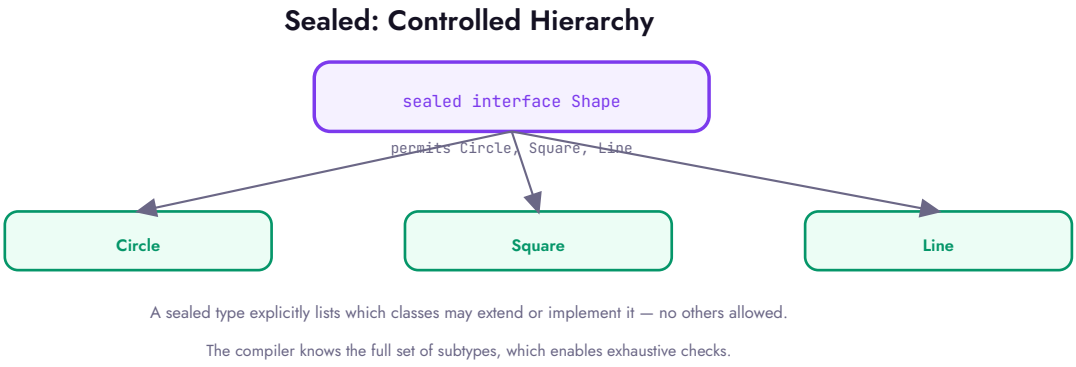
CHAPTER 09

Sealed Classes

Restricting which types can extend or implement a class or interface.

Java's inheritance was all-or-nothing: a class was either `final` (no subclasses) or open to anyone. **Sealed classes** (final in Java 17) add a middle ground — you list exactly which types may extend it.

A controlled hierarchy



A sealed type explicitly lists which classes may extend it — no others allowed.

```java
sealed interface Shape permits Circle, Square, Line {}

record Circle(double radius) implements Shape {}
record Square(double side)   implements Shape {}
record Line(double length)   implements Shape {}
```

Why restrict inheritance

- **Model a closed set** — a shape is a circle, square, or line, nothing else.
- **Exhaustiveness** — the compiler knows every subtype (key for pattern matching).
- **Safer APIs** — nobody can add an unexpected subtype to your hierarchy.

The permits clause

The `permits` clause names the allowed subtypes. If they live in the same file, you can even omit it — the compiler infers the permitted set from the declarations in that file.

Inheritance you control

A sealed class or interface lists exactly which types may extend it via permits. This models a closed set of subtypes, lets the compiler reason about exhaustiveness, and keeps your hierarchy from being extended unexpectedly.

© 2026 ByteMart Digital

MODULE 3 · SEALED CLASSES & PATTERN MATCHING

CHAPTER 10

Sealed Classes in Depth

The rules for permitted subtypes and the final/sealed/non-sealed choice.

Sealing a type comes with rules — every permitted subtype must declare how it continues the hierarchy. Understanding these three choices is the key to using sealed types correctly.

Every subtype must choose

Each class that extends a sealed type must itself be declared `final`, `sealed`, or `non-sealed` — the compiler requires it, so the hierarchy stays fully defined.

Modifier	Meaning
final	no further subclasses
sealed	continues sealing, with its own permits
non-sealed	reopens — anyone may extend this one

java

```
sealed interface Shape permits Circle, Polygon {}
final class Circle implements Shape {}          // closed
sealed class Polygon implements Shape
    permits Triangle, Rectangle {}              // continues
non-sealed class Rectangle extends Polygon {}   // reopened
```

Location rules

Permitted subtypes must be accessible to the sealed type — in the same module (or the same package for the unnamed module). This keeps the hierarchy contained and analysable.

Sealed plus records

The most common pattern is a **sealed interface permitting a set of records**. The interface fixes the variants, and each record models one — a concise, closed, immutable hierarchy. This sets up the pattern matching in the next chapters.

> **Every subtype declares its stance**
> Each permitted subtype of a sealed type must be final (closed), sealed (continues), or non-sealed (reopened), and must be accessible in the same module. Sealed interfaces permitting records are the idiomatic pattern.

MODULE 3 · SEALED CLASSES & PATTERN MATCHING

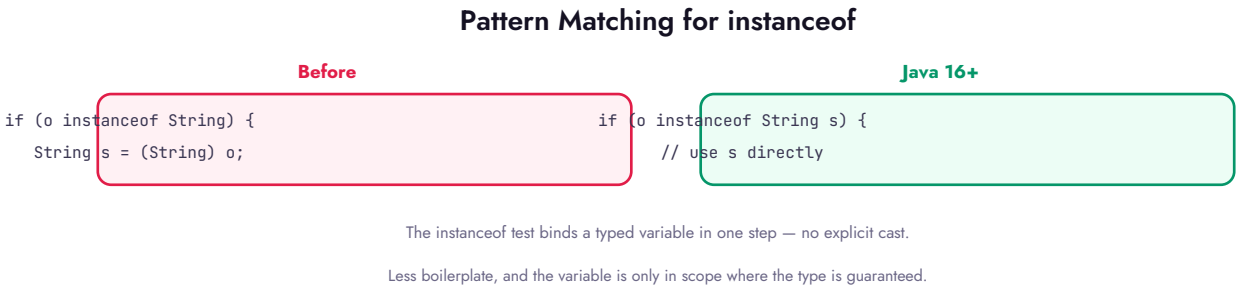
CHAPTER 11

Pattern Matching for instanceof

Testing a type and binding a typed variable in one clean step.

The old `instanceof`-then-cast dance was pure boilerplate: test the type, then cast to the same type. **Pattern matching for instanceof** (final in Java 16) collapses it into one step.

Before and after



The instanceof test binds a typed variable in one step — no explicit cast.

```java
// before
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}

// Java 16+
if (obj instanceof String s) {
    System.out.println(s.length());   // s already typed
}
```

The binding variable

When `obj instanceof String s` is true, `s` is bound to `obj` cast to `String`. No manual cast, and `s` is only in scope where the type is guaranteed.

Flow-sensitive scope

The binding works with the control flow. It is in scope in the `if` body, and you can even use it in the condition itself with && — the compiler tracks exactly where the type is known.

```java
if (obj instanceof String s && s.length() > 5) {
    // s is usable here — both conditions guarantee it
}
```

Test and bind at once

Pattern matching for instanceof binds a typed variable when the test passes — no separate cast. The binding is flow-sensitive, in scope only where the type is guaranteed, and usable within the condition itself.

MODULE 3 · SEALED CLASSES & PATTERN MATCHING

CHAPTER 12

Pattern Matching for switch

Branching on the type of a value with concise, exhaustive switch cases.

Chains of `if-else instanceof` are common but ugly. **Pattern matching for switch** lets a `switch` branch on a value's type, binding a variable per case — far cleaner, and exhaustive when combined with sealed types.

Switching on type

Pattern Matching for switch

```
String describe(Object o) {
     return switch (o) {
 case Integer i -> "int: " + i;
case String s  -> "text: " + s;
   default        -> "other";
           };
          }
```

switch can branch on the type (and shape) of a value, binding a variable per case.

Preview in Java 17, finalized in Java 21 — the successor to chains of if-instanceof.

switch can branch on the type of a value, binding a variable per case.

```java
String describe(Object o) {
    return switch (o) {
        case Integer i -> "int: " + i;
        case String s  -> "text of length " + s.length();
        case null      -> "null!";
        default        -> "something else";
    };
}
```

Guards

A `when` guard adds a condition to a case, so you can distinguish values of the same type — for example a positive versus negative integer — within the switch.

```java
case Integer i when i > 0 -> "positive";
case Integer i           -> "non-positive";
```

Preview in 17, final in 21

WATCH OUT

Pattern matching for switch is a **preview feature** in Java 17 — usable with `--enable-preview`, but finalized (with refinements) in Java 21. On Java 17 LTS, treat it as a look ahead; on 21 it is standard.

Exhaustiveness with sealed types

When you switch over a **sealed** type, the compiler knows every possible subtype. If your cases cover them all, no `default` is needed — and if you miss one, it is a compile error. This is the payoff of sealing.

Branch on type, exhaustively

Pattern matching for switch branches on a value's type, binding a variable per case, with when guards for extra conditions. Over a sealed type it is exhaustive and compiler-checked. It is preview in 17, final in 21.

MODULE 3 · SEALED CLASSES & PATTERN MATCHING

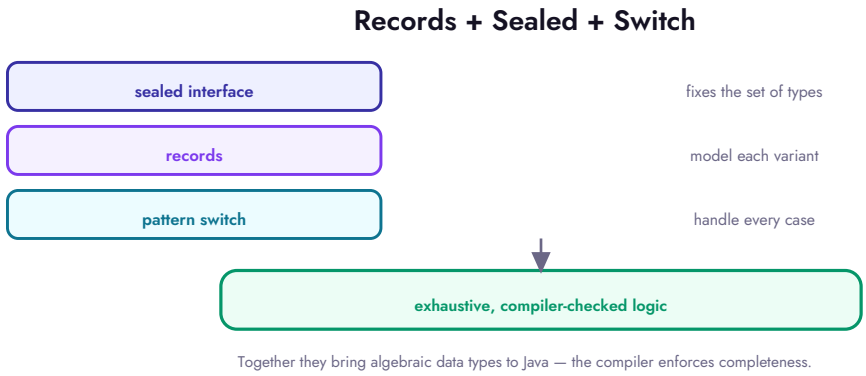
CHAPTER 13

Records + Sealed + Patterns

How three modern features combine into algebraic data types with compiler-checked completeness.

Records, sealed classes, and pattern matching are powerful alone — but together they bring **algebraic data types** to Java: a closed set of variants, each a concise record, handled by an exhaustive switch the compiler verifies.

The combination



Sealed fixes the set of types, records model each variant, and pattern switch handles every case.

```java
sealed interface Payment permits Card, Upi, Cash {}
record Card(String number) implements Payment {}
record Upi(String vpa)     implements Payment {}
record Cash()              implements Payment {}

String describe(Payment p) {
    return switch (p) {             // exhaustive: no default
        case Card c -> "card ending " + c.number();
        case Upi u  -> "upi " + u.vpa();
        case Cash c -> "cash";
    };
}
```

Why this is powerful

- **Sealed** fixes the complete set of variants.
- **Records** model each variant concisely and immutably.
- **Pattern switch** handles each, with no default needed.
- **The compiler** errors if you add a variant and forget to handle it.

Compiler-enforced completeness

Add a new `Payment` subtype and every exhaustive switch over `Payment` that does not handle it fails to compile. The compiler points you at every place that needs updating — a safety net that hand-written type

checks never gave you.

NOTE

Record patterns (destructuring a record directly in a case) arrive in Java 21, making this even more expressive. In 17 you switch on the type and call accessors.

Algebraic data types in Java

Sealed interfaces of records, matched by an exhaustive switch, give Java algebraic data types: a closed variant set, concise and immutable, with compiler-enforced completeness — forget a case and it will not compile.

MODULE 4 · SWITCH, COLLECTIONS & STREAMS

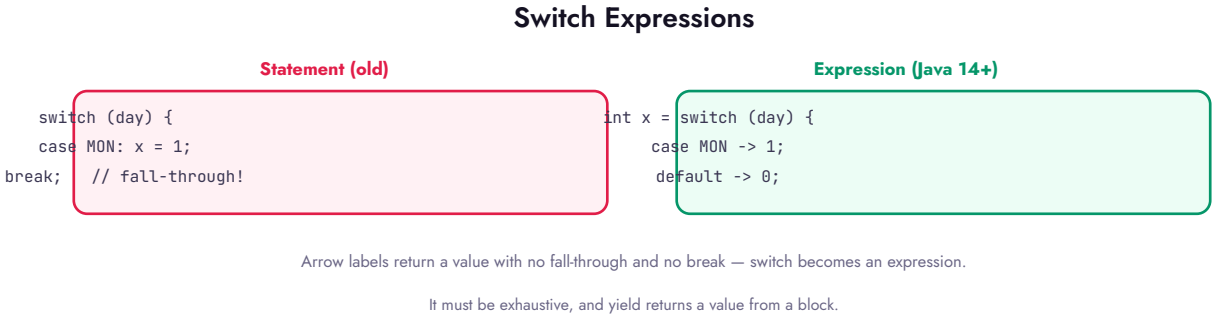
CHAPTER 14

Switch Expressions

The modern switch that returns a value, with no fall-through and no break.

The classic `switch` statement was a well-known source of bugs — forgotten `break`s and accidental fall-through. **Switch expressions** (final in Java 14) fix this and let switch return a value.

Statement vs expression



Arrow labels return a value with no fall-through and no break — switch becomes an expression.

```java
// old statement — fall-through, break needed
switch (day) {
    case MON: case TUE: n = 1; break;
    default: n = 0;
}

// expression — returns a value, no break
int n = switch (day) {
    case MON, TUE -> 1;
    default       -> 0;
};
```

The improvements

- **Arrow labels** (->) — no fall-through, no break.
- **Returns a value** — assign the whole switch to a variable.
- **Multiple labels** — `case MON, TUE` in one line.
- **Exhaustive** — must cover all cases (or have a default).

yield for blocks

When a case needs more than one statement, use a block and `yield` to return its value — the expression equivalent of a return for that branch.

```java
int size = switch (item) {
    case SMALL -> 1;
    case LARGE -> {
        log.info("large item");
        yield 3;
    }
};
```

switch that returns safely

Switch expressions use arrow labels with no fall-through or break, return a value you can assign, allow multiple labels per case, and must be exhaustive. Use yield to return a value from a multi-statement block.

MODULE 4 · SWITCH, COLLECTIONS & STREAMS

CHAPTER 15

Collection Factory Methods

Creating small immutable collections in one concise, readable call.

15

Creating a small constant collection in Java 8 was clumsy — construct, add, add, add. The **collection factory methods** (Java 9) create immutable lists, sets, and maps in a single call.

The factories

```java
List<String> roles = List.of("admin", "user");
Set<Integer> primes = Set.of(2, 3, 5, 7);
Map<String, Integer> ages = Map.of("Ada", 36, "Alan", 41);
// concise, and immutable
```

They are immutable

Collections from `List.of`, `Set.of`, and `Map.of` are **unmodifiable** — any attempt to add or remove throws `UnsupportedOperationException`. This makes them perfect for constants and defensive copies.

WATCH OUT

These factories reject `null` elements and, for sets and map keys, reject duplicates at creation time. They are stricter than the old collections — usually a good thing, but a change to be aware of.

Copying immutably

`List.copyOf(existing)` (Java 10) makes an immutable snapshot of another collection — ideal for returning data from a method without exposing your internal mutable list.

```java
return List.copyOf(this.items);   // caller cannot mutate your state
```

Immutable collections, one call

List.of, Set.of, and Map.of create small immutable collections concisely; copyOf snapshots an existing one. They reject nulls and duplicates, making them ideal for constants and safe returns.

© 2026 ByteMart Digital

MODULE 4 · SWITCH, COLLECTIONS & STREAMS

CHAPTER 16

Stream Enhancements

The stream and collector additions that landed after Java 8.

Streams arrived in Java 8, but later versions polished them. A handful of additions make common stream tasks cleaner — worth knowing if streams are part of your daily code.

Stream.toList()

Collecting to a list was verbose: `.collect(Collectors.toList())`. Java 16 added `Stream.toList()` — shorter, and it returns an unmodifiable list.

```java
// Java 8
List<String> names = users.stream()
    .map(User::name).collect(Collectors.toList());

// Java 16+
List<String> names = users.stream().map(User::name).toList();
```

Other additions

Addition	Does
Stream.toList()	collect to an unmodifiable list (16)
Stream.mapMulti	flat-map imperatively (16)
Collectors.teeing	combine two collectors (12)
Stream.ofNullable	0 or 1 element from a value (9)
takeWhile / dropWhile	prefix-based slicing (9)

teeing

`Collectors.teeing` feeds the stream to two collectors and merges their results — for example computing a sum and a count in one pass to get an average.

takeWhile and dropWhile

`takeWhile` takes elements until a predicate fails; `dropWhile` skips them until it fails. On ordered streams they slice by a leading condition — cleaner than filtering plus state.

Streams, refined

Post-8 streams added toList() (unmodifiable, concise), mapMulti, Collectors.teeing (two collectors in one pass), ofNullable, and takeWhile/dropWhile. Small additions that simplify everyday stream code.

© 2026 ByteMart Digital

MODULE 4 · SWITCH, COLLECTIONS & STREAMS

CHAPTER 17

Optional Improvements

The methods that make Optional more expressive and less clumsy.

`Optional` arrived in Java 8 to tame null, but its early API was limited. Later versions added methods that make it genuinely pleasant — and reduce the temptation to call `get()`.

The key additions

Method	Does
ifPresentOrElse	run one of two actions (9)
or	supply a fallback Optional (9)
stream	0-or-1-element Stream (9)
orElseThrow()	no-arg, throws if empty (10)

ifPresentOrElse

java

```
user.ifPresentOrElse(
    u -> log.info("found {}", u.name()),
    () -> log.warn("no user")
);
```

Chaining fallbacks with or

`or` lets you try alternatives lazily — look in the cache, then the database — each returning an `Optional`, without nesting `isPresent` checks.

java

```
var result = fromCache(id).or(() -> fromDatabase(id));
```

orElseThrow

The no-arg `orElseThrow()` (Java 10) is the clean way to assert a value is present — it throws `NoSuchElementException` if empty. Prefer it over `get()`, which reads as less intentional.

> **Optional, more expressive**
> Later methods make Optional fluent: ifPresentOrElse for two-way handling, or for lazy fallbacks, stream to bridge into streams, and no-arg orElseThrow to assert presence — use it instead of get().

MODULE 4 · SWITCH, COLLECTIONS & STREAMS

CHAPTER 18

Helpful NullPointerExceptions

How Java 14 turned the most common exception into an actually useful message.

The `NullPointerException` is Java's most common exception, and its message used to be uselessly vague. Java 14's **helpful NPEs** tell you exactly which variable was null.

The old problem

A chained expression that threw an NPE gave only a line number — but a line like `a.getB().getC().getD()` has several things that could be null, and you were left guessing which.

The helpful message

```text
Before:
  Exception in thread "main" java.lang.NullPointerException

Java 14+:
  Cannot invoke "C.getD()" because the return value of
  "B.getC()" is null
```

The new message names the exact expression that was null and what you were trying to do with it — turning a guessing game into a direct pointer to the bug.

On by default

Helpful NPEs are enabled by default from Java 15 onward (opt-in in 14). You get better debugging for free, with no code changes — one of the quiet quality-of-life wins of modern Java.

TIP

When you see one of these messages, read it literally: the phrase after 'because' is the sub-expression that evaluated to null. It usually points straight at the fix.

NPEs that name the culprit

Helpful NullPointerExceptions (Java 14, default from 15) state exactly which sub-expression was null and what you tried to do with it — turning Java's most common exception from a guessing game into a direct clue.

MODULE 5 · PLATFORM FEATURES

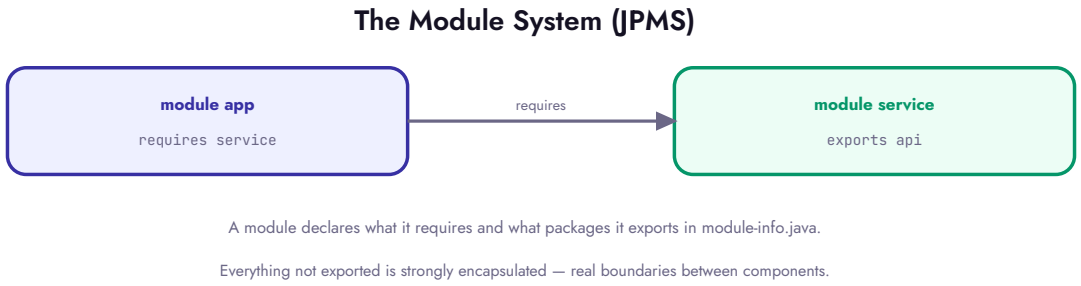
CHAPTER 19

The Module System (JPMS)

Java's system for strong encapsulation and explicit dependencies between components.

The **Java Platform Module System** (JPMS), introduced in Java 9, lets you group packages into **modules** that explicitly declare what they need and what they expose — real boundaries beyond the classpath.

Requires and exports



A module declares what it requires and what packages it exports in module-info.java.

```java
// module-info.java
module com.bytemart.orders {
    requires com.bytemart.common;   // depends on
    exports com.bytemart.orders.api;  // makes public
    // everything else stays hidden
}
```

What modules give you

- **Strong encapsulation** — non-exported packages are truly hidden, even via reflection.
- **Explicit dependencies** — `requires` makes the dependency graph clear.
- **Reliable configuration** — missing modules fail fast at startup, not at runtime.

Should you modularise?

WATCH OUT

JPMS is powerful but adds complexity, and many applications run fine on the classpath without module declarations. The JDK itself is modularised (which is why encapsulation of internals matters), but modularising your own app is optional — adopt it when you need strict boundaries, not by default.

You benefit either way

Even if you never write a `module-info.java`, the JDK's own modularisation shapes modern Java — it is why internal APIs are now encapsulated (the next chapters) and why the platform is leaner.

Explicit, encapsulated components

JPMS groups packages into modules that declare requires and exports in module-info.java, giving strong encapsulation and an explicit dependency graph. Adopt it for strict boundaries — many apps run fine on the classpath.

MODULE 5 · PLATFORM FEATURES

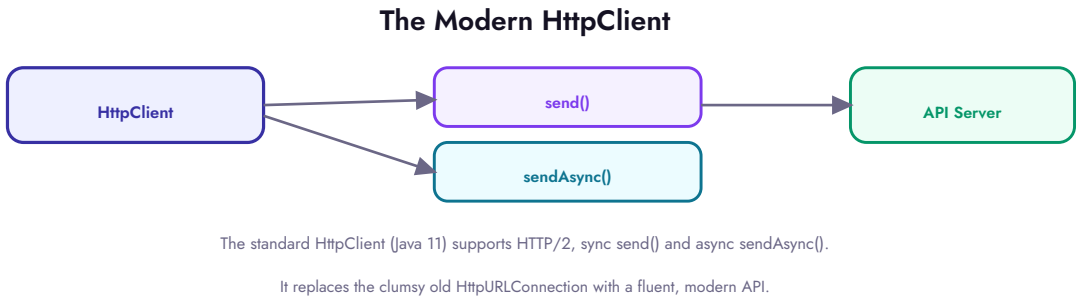
CHAPTER 20

The Modern HttpClient

The standard HTTP client that replaced HttpURLConnection.

Making HTTP calls in Java used to mean the awkward `HttpURLConnection` or a third-party library. Java 11 made the modern **HttpClient** standard — fluent, HTTP/2-capable, and both synchronous and asynchronous.

Sync and async



The standard HttpClient supports HTTP/2, synchronous send() and asynchronous sendAsync().

```java
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.bytemart.com/orders"))
    .header("Accept", "application/json")
    .GET()
    .build();

HttpResponse<String> res = client.send(request,
    HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());
System.out.println(res.body());
```

Asynchronous calls

`sendAsync` returns a `CompletableFuture`, so you can make non-blocking calls and compose them — fire several requests and handle each result as it arrives, without tying up a thread each.

```java
client.sendAsync(request, BodyHandlers.ofString())
      .thenApply(HttpResponse::body)
      .thenAccept(System.out::println);
```

Why it is better

- **HTTP/2** support out of the box, falling back to 1.1.
- **Fluent builders** for requests and clients.
- **Sync and async** from one API.
- **No dependency** — it is part of the JDK.

A real HTTP client, built in

The standard HttpClient (Java 11) offers a fluent, HTTP/2-capable API with synchronous send() and asynchronous sendAsync() returning a CompletableFuture — replacing HttpURLConnection with no external dependency.

MODULE 5 · PLATFORM FEATURES

CHAPTER 21

Enhanced Random & API Additions

The pseudo-random generator overhaul and other smaller library wins.

Beyond the headline features, modern Java quietly improved many corners of the library. The random-number overhaul in Java 17 is a good example, alongside several smaller additions.

Enhanced random generators

Java 17 introduced the `RandomGenerator` interface, unifying the old `Random` with a family of new, higher-quality algorithms selectable by name — useful for simulations and anywhere the old generator's statistical quality was not enough.

```java
RandomGenerator rng =
    RandomGeneratorFactory.of("L64X128MixRandom").create();
int roll = rng.nextInt(1, 7);   // 1..6
```

Files made easy

`Files.readString` and `Files.writeString` (Java 11) read or write an entire file's text in one call — no streams, readers, or boilerplate for the common case.

```java
String content = Files.readString(Path.of("data.json"));
Files.writeString(Path.of("out.txt"), "hello");
```

Other conveniences

- **Predicate.not** (Java 11) — negate a method reference cleanly.
- **String.formatted** — format on the instance, great with text blocks.
- **Teeing, mapMulti**, and the stream additions from earlier.

Quiet library wins

Java 17's RandomGenerator interface unifies random with better algorithms; Files.readString/writeString handle whole-file text in one call; and helpers like Predicate.not smooth everyday code.

© 2026 ByteMart Digital

MODULE 5 · PLATFORM FEATURES

CHAPTER 22

Strong Encapsulation of Internals

Why JDK internal APIs are now locked down — and what it means for your code.

For years, code reached into JDK internals — `sun.misc.Unsafe`, private fields via reflection. Modern Java, and Java 17 in particular, **strongly encapsulates** these internals, which is a common migration surprise.

The change

Java 16 turned on strong encapsulation by default, and Java 17 removed the `--illegal-access` escape hatch. Reflective access to internal JDK APIs that used to print a warning now **fails**. Your own code is unaffected; only access to JDK internals is restricted.

WATCH OUT

This is the migration issue most likely to bite: an old library that reflects into JDK internals may throw `InaccessibleObjectException` on Java 17. The fix is almost always to update the library to a version that does not do this.

Why lock it down

- **Security** — internal APIs were never meant to be public.
- **Maintainability** — the JDK can evolve internals without breaking callers.
- **Correctness** — internals had no compatibility guarantees.

If you truly need access

As a last resort, the `--add-opens` flag can reopen a specific package at runtime — but treat it as a temporary bridge while you remove the dependency on internals, not a permanent solution.

Internals are now off-limits

Java 17 strongly encapsulates JDK internals — reflective access that once warned now fails. It improves security and lets the JDK evolve. The usual fix on migration is updating old libraries; –add-opens is a temporary bridge.

© 2026 ByteMart Digital

MODULE 6 · MIGRATION & PRACTICE

CHAPTER 23

Migrating from Java 8 to 17

The practical gotchas of the big upgrade, and how to work through them.

Upgrading from Java 8 to 17 is usually smoother than feared — the language is backward compatible — but a handful of removals and restrictions catch teams out. Knowing them turns a scary migration into a checklist.

Removed Java EE modules

Java 11 removed the bundled Java EE and CORBA modules — JAXB, JAX-WS, and related APIs. Code that used `javax.xml.bind` now needs those libraries added as explicit dependencies (the Jakarta EE artifacts).

The common migration issues

Issue	Fix
JAXB/JAX-WS missing	add jakarta dependencies
reflective access fails	update the offending library
Nashorn (JS) removed	use a scripting library
removed GCs / flags	update JVM options

A migration order

- **Update build and dependencies** first — many issues vanish with current libraries.
- **Compile on 17** and fix warnings and errors.
- **Run the test suite** — watch for reflective-access failures.
- **Review JVM flags** — some GCs and options were removed.
- **Adopt features gradually** — migrate first, modernise after.

WATCH OUT

Do not try to rewrite everything into records and switch expressions during the migration itself. Get it **running** on 17 first with minimal changes, then modernise the code in a separate, calmer pass.

Migrate first, modernise later

The Java 8-to-17 jump is mostly compatible; the snags are removed EE modules (add Jakarta deps), blocked reflective access (update libraries), and removed flags. Update dependencies, get it running, then adopt new features separately.

© 2026 ByteMart Digital

MODULE 6 · MIGRATION & PRACTICE

CHAPTER 24

Best Practices & Modern Java Style

Habits for writing idiomatic, modern Java once you are on 17.

Being on Java 17 is one thing; writing modern Java is another. This closing chapter distils the habits that make the most of everything the guide has covered.

Prefer the modern idioms

- **Records** for immutable data instead of boilerplate classes.
- **Sealed types + pattern switch** to model and handle closed variant sets.
- **Switch expressions** over statement switches with break.
- **Pattern instanceof** instead of test-then-cast.
- **var** for local variables where the type is obvious.
- **Text blocks** for embedded JSON, SQL, and HTML.

Prefer immutability

Modern Java leans toward immutability — records, `List.of`, `toList()`, and unmodifiable returns. Immutable data is easier to reason about and safe to share across threads. Reach for it by default.

Use the better library methods

Small choices add up: `strip` over `trim`, `toList()` over the old collector, the no-arg `orElseThrow` over `get`, `HttpClient` over `HttpURLConnection`, and `Files.readString` over manual streams.

Modernise deliberately

You do not have to rewrite everything at once. As you touch old code, nudge it toward the modern idioms — a record here, a switch expression there. Over time the codebase becomes cleaner without a risky big-bang rewrite.

> **Write Java the modern way**
> On Java 17, prefer records, sealed types with pattern switch, switch expressions, pattern instanceof, var, and text blocks. Lean toward immutability and the newer library methods, and modernise code gradually as you touch it.

© 2026 ByteMart Digital